

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №6
По дисциплине «Современные платформы программирования»
Тема: «Phyton тестирование»**

Выполнил:
Студент 3 курса
Группы ПО-12
Боричевский Д. О.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

- 1) Изложение цели работы.
- 2) Задание по лабораторной работе с описанием своего варианта.
- 3) Спецификации ввода-вывода программы.
- 4) Текст программы (кратко).
- 5) Выводы по проделанной работе.

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:
 - Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
 - Проверка выброса ошибки при отрицательной цене.
 - Проверка вычисления общей стоимости (`total()`).
2. Протестируйте метод `apply_discount` с разными значениями скидки:
 - 0% - цена остаётся прежней
 - 50% - цена уменьшается вдвое
 - 100% - цена становится ноль
 - < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`

```
@pytest.fixture  
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

Код программы:

```
"""
```

Мини-библиотека покупок для тестирования.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import Dict
```

```
@dataclass
```

```
class Item:
```

```
    name: str
```

```
    price: float
```

```
class Cart:
```

```
    """Корзина покупок."""
```

```
    def __init__(self) -> None:
```

```
        self.items: Dict[str, float] = {}
```

```
    def add_item(self, name: str, price: float) -> None:
```

```
        if price < 0:
```

```
            raise ValueError("Price cannot be negative")
```

```
        self.items[name] = price
```

```
    def total(self) -> float:
```

```
        return sum(self.items.values())
```

```
    def apply_discount(self, percent: float) -> None:
```

```
if percent < 0 or percent > 100:

    raise ValueError("Invalid discount")

factor = 1 - percent / 100

for name in list(self.items):

    self.items[name] = round(self.items[name] * factor, 2)
```

```
def log_purchase(item: dict) -> None:

    """

    Логирование покупки в удалённую систему.

    В testmax requests.post будет замочан.

    """

    import requests

    requests.post("https://example.com/log", json=item)
```

```
def apply_coupon(cart: Cart, coupon_code: str) -> None:

    coupons = {"SAVE10": 10, "HALF": 50}

    if coupon_code not in coupons:

        raise ValueError("Invalid coupon")

    cart.apply_discount(coupons[coupon_code])
```